

React Testing Library

[React Testing Library](#) помогает вам писать тесты, независимые от деталей реализации. Для этого она предоставляет методы для поиска элементов, а также пару хелперов для упрощения работы с асинхронными операциями (`waitFor` и `waitForElementToBeRemoved`)

Поиск элементов

Для поиска элементов существуют следующие семейства методов:

Тип запроса	0 совпадений	1 совпадение	>1 совпадения	>Повтор
1 элемент				
<code>getBy...</code>	Ошибка	Возвращает элемент	Ошибка	Нет
<code>queryBy...</code>	Возвращает null	Возвращает элемент	Ошибка	Нет
<code>findBy...</code>	Ошибка	Возвращает элемент	Ошибка	Да
Несколько элементов				
<code>getAllBy...</code>	Ошибка	Возвращает массив	Возвращает массив	Нет
<code>queryAllBy...</code>	Возвращает []	Возвращает массив	Возвращает массив	Нет
<code>findAllBy...</code>	Ошибка	Возвращает массив	Возвращает массив	Да

В большинстве случаев вы будете пользоваться двумя семействами:

- [*ByText](#) - ищет по тексту внутри элемента;
- [*ByTestId](#) - ищет по атрибуту `data-qa`.

Поиск внутри элемента

Для поиска внутри элемента есть метод [within](#).

```
1 const messages = document.getElementById('messages');
2 const helloMessage = within(messages).getByText('hello');
```

Ожидание асинхронного действия

Часто после выполнения какого-то действия нужно будет подождать пока оно применится. В этом поможет метод [waitFor](#):

```
1 await waitFor(() => {
2   expect(listView.items()).toHaveLength(1);
3 });
```

Этот метод вызывает коллбек с определённым интервалом, пока тот не перестанет кидать исключение.

Ожидание удаления элемента

Чтобы подождать пока элемент удалится из DOM можно использовать метод [waitForElementToBeRemoved](#).

```
1  await  
   waitForElementToBeRemoved(document.querySelector('div.getOuttaHere'));
```